

ICS小班课回课-系统级IO

王嘉怡 2400013107

2025.11.26

Standard I/O

更高级别的抽象，将每一个打开的文件模型化为一个指向 `FILE` 类型结构体的指针（流）。

(1) 基于课件第33~34页，说明列出的每个标准I/O函数的功能；说明三个默认打开的I/O文件的作用，它们和键盘、显示屏幕之间是什么关系。

标准I/O函数

函数	功能	举例	解释
<code>fopen</code>	打开文件	<code>FILE *file = fopen("data.txt", "w");</code>	以写入模式 ("w") 打开名为 <code>data.txt</code> 的文件，并返回文件指针。
<code>fclose</code>	关闭文件	<code>int result = fclose(file);</code>	关闭之前由 <code>fopen</code> 打开的 文件指针 <code>file</code> 。
<code>fread</code>	读字节	<code>size_t bytes_read = fread(buffer, sizeof(char), 100, file);</code>	从 <code>file</code> 读取 100 个大小为 <code>sizeof(char)</code> (1 字节) 的数据块，存入 <code>buffer</code> 。
<code>fwrite</code>	写字节	<code>size_t bytes_written = fwrite(data, sizeof(int), 5, file);</code>	将 <code>data</code> 数组中 5 个大小为 <code>sizeof(int)</code> 的数据块写入 <code>file</code> 。
<code>fgets</code>	读字符串	<code>char *line = fgets(buffer, 128, file);</code>	从 <code>file</code> 读取至多 127 个字符（留一个给空字符 <code>\0</code> ），存入 <code>buffer</code> 。

函数	功能	举例	解释
fputs	写字符串	int status = fputs("Hello World\n", file);	将字符串 "Hello World\n" 写入 file。
scanf	格式化输入	int num_read = scanf("%d %s", &number, name);	从标准输入 (stdin) 读取一个整数和一个字符串，分别存入 number 和 name。
printf	格式化输出	int chars_printed = printf("The result is: %d\n", result_value);	将格式化后的字符串和整数 result_value 打印到标准输出 (stdout)
fscanf	从文件流中读取格式化数据	int num_read = fscanf(file, "%d %s", &age, city);	从文件指针 file 所指向的文件中读取一个整数和一个字符串。
fprintf	向文件流中写入格式化数据	int chars_printed = fprintf(file, "User ID: %d, Name: %s\n", id, name);	将格式化字符串和变量 id、name 写入到文件指针 file 所指向的文件中。

- 注: `scanf(...)` $\approx$ `fscanf(stdin, ...)` 从**标准输入**读取; `printf(...)` $\approx$ `fprintf(stdout, ...)` 写入到**标准输出**。

核心变化: 接口从 `int` 类型的文件描述符, 变成了 `FILE*` 类型的文件指针, 包含了文件描述符和流缓冲区。

默认打开的I/O文件

```

1  #include <stdio.h>
2  extern FILE *stdin; /* standard input (descriptor 0) */
3  extern FILE *stdout; /* standard output (descriptor 1) */
4  extern FILE *stderr; /* standard error (descriptor 2) */

```

- 标准输入 `stdin`: 从键盘中读取输入数据, 相当于**键盘文件的抽象**
- 标准输出 `stdout`: 将数据写到显示屏幕, 相当于**显示屏幕文件的抽象**

- 标准错误 `stderr`：将错误和诊断信息写到显示屏幕，相当于**显示屏幕文件的抽象**

Buffered I/O

Motivation

(2) 基于课件第35~36页，说明I/O操作为什么要用带缓冲的形式。

空间换时间

- 每次 `read` 或 `write` 读写操作的时候都需要**系统调用**，从用户模式切换到内核模式很耗时 ($> 10,000$ clock cycles)。使用缓冲，可以将多次小的I/O合并成一次或几次大的I/O。这样就**显著减少了用户/内核模式的切换次数**，从而节省了开销。
- 磁盘、网络等硬件设备的I/O速度相对于CPU和内存的操作速度来说非常慢。每次直接读写硬件（例如磁盘）都需要经历一个耗时的过程，缓冲一块内存区域充当了程序和硬件之间的**高速缓存**。
- 方便处理不同大小的数据，使用起来更加方便灵活
eg. 计算文件文本行的数量：一个一个字节读取并检查换行符 vs 一行一行读取

Action

(3) 基于课件第37~38页，讲解程序示例，解释I/O相关的操作在C程序层面和实际执行层面的差异，并分析原因。

```

1  #include <stdio.h>
2  int main() {
3      printf("h");
4      printf("e");
5      printf("l");
6      printf("l");
7      printf("o");
8      printf("\n");
9      fflush(stdout); // 强制清空缓冲区
10     exit(0);
11 }
```

用 `strace` 来跟踪系统调用：

```

1  linux> strace ./hello
2  execve("./hello", ["hello"], [/ * ... */]).
3  ...
4  write(1, "hello\n", 6)           = 6 # 返回值，表示成功写入六个字
   符
5  ...
6  exit_group(0)                   = ? # 表示系统调用不会返回

```

- 调用 `printf("h");` 时，并**不立即**调用 `write`，只是快速地将字符 'h' 复制到内存缓冲区中。后续的 'e', 'l', 'l', 'o' 也被追加到缓冲区中。此时没有发生任何系统调用。
- 当**缓冲区满了/遇到换行符/显式调用 `fflush()` /程序退出**时，缓冲区中的数据才会触发 `write` 系统调用，一次性传给内核，（`write(1, "hello\n", 6)`）。

注：`= ?` 是 `strace` 用来表示这个系统调用**不会返回**（因为它终止了整个进程）的一种特殊标记。

RIO Package

RIO函数

(5) 基于课件第43~49页，逐个简要介绍RIO函数提供的功能。

Unbuffered RIO Input and Output

函数	功能	返回
<code>ssize_t rio_readn(int fd, void *usrbuf, size_t n)</code>	从描述符 <code>fd</code> 的当前文件位置 最多传送 n 个字节 到内存位置 <code>usrbuf</code>	成功：读的字节数（遇到 EOF 为不足值）； EOF：0；出错：-1

函数	功能	返回
<code>ssize_t rio_writen(int fd, void *usrbuf, size_t n)</code>	从位置 <code>usrbuf</code> 传送 <code>n</code> 个字节到描述符 <code>fd</code>	成功：写的字节数； 出错：-1； 绝不返回不足值

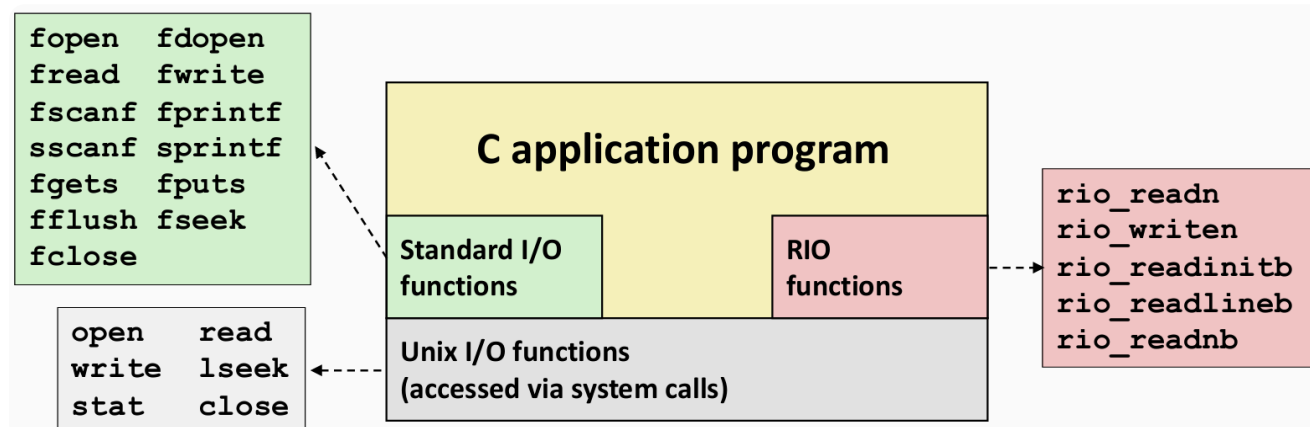
Buffered RIO Input Functions

函数	功能	返回
<code>void rio_readinitb(rio_t *rp, int fd)</code>	打开一个描述符都会调用。将 <code>fd</code> 与 <code>*rp</code> 缓冲区联系起来。	-
<code>ssize_t rio_readlineb(rio_t *rp, void *usrbuf, size_t maxlen)</code>	从文件 <code>rp</code> 读取 下一行 文本（包括结尾 <code>\n</code> ）复制到 <code>usrbuf</code> ，最多 <code>maxlen-1</code> 字节，超过部分被截断。	成功：读的字节数； EOF：0； 出错：-1
<code>ssize_t rio_readnb(rio_t *rp, void *usrbuf, size_t n)</code>	从文件 <code>rp</code> 读取 最多n字节 到 <code>usrbuf</code>	成功：读的字节数； EOF：0； 出错：-1

调用可以任意交叉进行，但是**不能和 `rio_readn` 交叉使用。

Unix I/O vs. Standard I/O vs. Robust I/O

（4）基于课件第40~42页，讲解C标准I/O、Unix I/O和RIO三种方式在软件系统中所处的层次，并简要介绍各自的代表函数。



- Unix I/O是应用程序与系统内核进行I/O交互的唯一入口点，处于I/O系统的最底层。
- Standard I/O是建立在Unix I/O之上的一层软件封装。
- Rubust I/O是教材提供的一套应用级封装库，提供无缓冲的输出输出函数和带缓冲的输入函数。

(6) 基于课件第51~54页，对比说明C标准I/O、UnixI/O和RIO三种方式的优缺点和建议使用场景。

1. Unix I/O

• 优点

- 最通用，开销最低。所有其他I/O软件包都是基于Unix I/O实现的。
- 提供了访问文件元数据的函数。
- 异步信号安全，可以安全地在信号处理程序中使用。

• 缺点

- 不足值问题
- 读取文本行效率低下，除非有某种形式的缓冲

• 建议使用场景

- 在信号处理程序中。
- 在需要绝对最高性能的极少数情况下。

2. C标准 I/O (Standard I/O)

• 优点

- 缓冲减少了 read 和 write 系统调用的次数，提高了效率。
- 自动处理不足值。

• 缺点

- 不提供访问文件元数据的函数。
- 不是异步信号安全的，因此不适合在信号处理程序中使用。
- 标准I/O不适合用于网络套接字的输入和输出，（因为对流限制和对套接字的限制有时会相互冲突）。

• 建议使用场景

- 尽量使用标准I/O。对于磁盘和终端设备I/O*是首选。
- scanf 不能用来读二进制文件。

- 不适合用于网络套接字I/O。

3. RIO (Robust I/O)

- **优点**
 - 安全，鲁棒，抗信号中断。
 - 自动处理不足值。
 - 提供带缓冲读版本，行读取效率高。
- **缺点**
 - CSAPP自定义的。
- **使用场景**
 - 网络套接字I/O
 - `rio_readlineb` 不能用来读二进制文件。

应用/程序实例

(7) 基于课件第55~58页，逐个讲解程序示例。

I/O重定向

```

1  #include "csapp.h"
2  int main(int argc, char *argv[]) {
3      int fd1, fd2, fd3;
4      char c1, c2, c3;
5      char *fname = argv[1];
6      fd1 = Open(fname, O_RDONLY, 0);
7      fd2 = Open(fname, O_RDONLY, 0);
8      fd3 = Open(fname, O_RDONLY, 0);
9      Dup2(fd2, fd3); // redirection
10     Read(fd1, &c1, 1);
11     Read(fd2, &c2, 1);
12     Read(fd3, &c3, 1);
13     printf("c1 = %c, c2 = %c, c3 = %c\n", c1, c2, c3);
14     return 0; }
15 // file: "abcde"
```

`fd2` 与 `fd3` 指向了同一个文件表项。

输出: `c1 = a, c2 = a, c3 = b`

进程间读共享文件

```

1  #include "csapp.h"
2  int main(int argc, char *argv[]) {
3      int fd1;
4      int s = getpid() & 0x1; // arbitrary
5      char c1, c2;
6      char *fname = argv[1];
7      fd1 = Open(fname, O_RDONLY, 0);
8      Read(fd1, &c1, 1);
9      if (fork()) { /* Parent */
10         sleep(s);
11         Read(fd1, &c2, 1);
12         printf("Parent: c1 = %c, c2 = %c\n", c1, c2); }
13     else { /* Child */
14         sleep(1-s);
15         Read(fd1, &c2, 1);
16         printf("Child: c1 = %c, c2 = %c\n", c1, c2); }
17     return 0; }
18 // file: "abcde"

```

父进程与子进程的 `fd1` 指向同一个文件表项。

- `s=1` : 子进程先执行。

输出:

```

1  Child: c1 = a, c2 = b
2  Parent: c1 = a, c2 = c

```

- `s=0` : 父进程先执行。

输出:

```

1  Parent: c1 = a, c2 = b
2  Child: c1 = a, c2 = c

```


共享文件的写

```

1  #include "csapp.h"
2  int main(int argc, char *argv[]) {
3      int fd1, fd2, fd3;
4      char *fname = argv[1];
5      fd1 = Open(fname, O_CREAT|O_TRUNC|O_RDWR, S_IRUSR|S_IWUSR);
6      Write(fd1, "pqrs", 4);
7      fd3 = Open(fname, O_APPEND|O_WRONLY, 0);
8      Write(fd3, "jklmn", 5);
9      fd2 = dup(fd1); /* Allocates descriptor */
10     Write(fd2, "wxyz", 4);
11     Write(fd3, "ef", 2);
12     return 0; }

```

注意 `fd1` 设置为 `O_TRUNC`，即打开的时候会截断文件（从零开始写），而 `fd2` 设置为 `O_APPEND`，即打开的时候从文件末尾开始写入。

从而文件写入过程：

次数	文件内容	fd1 指向位置	fd2 指向位置	fd3 指向位置
1	pqrs	4	-	-
2	pqrsjklmn	4	-	9
3	pqrswxyzn	8	8	9
4	pqrsxyznef	8	8	11

最终文件内容： pqrsxyznef

↓ 研讨题发之前做的一些内容（10.6 & 10.7）

读取文件/目录的信息

10.6 读取文件元数据

```

1  #include <unistd.h>

```

```

2  #include <sys/stat.h>
3
4  int stat(const char *filename, struct stat *buf);
5  int fstat(int fd, struct stat *buf);

```

- **功能**：输入文件名 `filename` 或文件描述符 `fd`，将文件元数据填写进 `buf` 指向的 `stat` 结构体中。
- **返回**：成功 `0`，失败 `-1`。

`stat` 中有用的元数据：

- `off_t st_size`：文件的字节数大小
- `mode_t st_mode`：文件访问许可位和文件类型
 - 通过下面三个宏谓词得知文件类型：
 - `S_ISREG(stat.st_mode)`：is regular? 是普通文件吗?
 - `S_ISDIR(stat.st_mode)`：is directory? 是目录文件吗?
 - `S_ISSOCK(stat.st_mode)`：is socket? 是网络套接字吗?
 - 通过和文件访问许可位 `&` 运算可以得知是否有对应权限
 - eg. `stat.st_mode & S_IRUSR`：`true` → 使用者能够读这个文件

10.7 读取目录内容

```

1  #include <sys/types.h>
2  #include <dirent.h>
3
4  DIR *opendir(const char *name);

```

- **功能**：输入 `name` 路径名，返回指向目录流（目录项列表）的指针。
- **返回**：成功为该指针，失败为 `NULL`。

```

1  #include <dirent.h>
2
3  struct dirent *readdir(DIR* dirp);

```

- **功能**：输入目录流指针 `dirp`，返回指向 `dirp` 中下一个目录项的指针。

- 目录项类型为 `dirent` 结构体，包含 `ino_t d_ino` 记录文件位置，`char d_name[256]` 记录文件名。
- **返回**：成功为该指针，*没有更多目标项或出错*为 `NULL`。
 - 区分错误和结束的唯一方式是查看 `errno`

```
1  #include <dirent.h>
2
3  int closedir(DIR *dirp);
```

- **功能**：关闭流并释放所有资源。
- **返回**：成功为 `0`，错误为 `-1`。